

DOI: 10.1145/1562164.1562200

BY DINESH BATRA

Modified Agile Practices for Outsourced Software Projects

FRUSTRATION WITH THE BUREAUCRATIC NATURE OF the disciplined approach has led to the call for agile development.³ The new approach is defined by the Agile Manifesto (<http://agilemanifesto.org/>), which values individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and agility in responding to change over following a prescribed plan. Agile development does not focus on process improvement; instead it focuses on customer satisfaction and employee empowerment. This is evident from reading the stated values and principles of the Agile Manifesto, which include fairly extreme positions such as “welcome changing requirements, even late in development” and “the best architectures, requirements, and designs emerge from self-organizing teams.”

An interesting issue arising from the call for agile development is its role in distributed development, which usually translates to offshore development.

A recent study⁷ indicates that agile practices can reduce temporal, geographical, and socio-cultural distances in distributed development projects. The study researched agile development between teams located in the U.S. and Ireland, and while it reported that overall communication was improved, it also noted problems related to geographical, temporal, and even language distances. Although there are other reported successes of distributed agile development,⁹ the projects are generally small, the team members are likely familiar with each other, and the participants are largely experts or high caliber developers.

This raises a research, as well as a practical, question: can we extend the use of agile practices from small projects to medium and large projects that involve a significant outsourcing component? To address this question, we must drop constraints such as small size projects, and expert developers belonging to the same company, and examine problems arising from geographical, temporal, and cultural distances. Accordingly, agile practices may need to be modified.

In this article, the key issues of software projects with an outsourced component are first identified. These issues are then used as a background to evaluate how standard agile practices stand up when applied to larger projects. This evaluation is followed by recommendations for modified agile practices for outsourced software projects.

Key Issues of Outsourced Software Projects

Although outsourcing is not equivalent to offshoring a software development project, the high availability of skilled labor and the economic advantages of considerably lower wages have made the two equivalent for all practical purposes. Outsourcing seems to have a stable future;^{4,5} several studies (www.acm.org/globalizationreport/) have shown that it improves productivity and does not necessarily lead to loss of jobs for the local region of the outsourcing cli-

ent. A severe recession has, indeed, recently led to massive layoffs; however, these layoffs have primarily resulted from a weak economy, not because of outsourcing. The primary motive for offshoring is to keep costs down, but there are several other noteworthy benefits, such as the ability to undertake larger projects, to expand markets,¹¹ and to achieve agility in development and operation.⁵ However, offshoring does not always lead to cost savings or more successful projects.

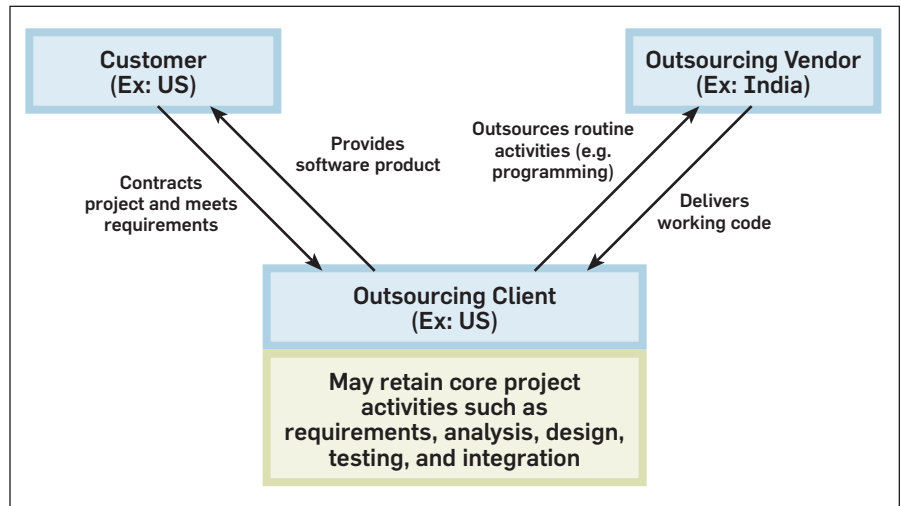
When a software development project is outsourced, generally three parties are involved (Figure 1): the customer, the outsourcing client, and the outsourcing vendor. Consider a typical scenario: a customer in the US needs an information system, and a software division in the customer's same company or another software development company in the U.S. is asked to develop the system. The outsourcing client is the primary software development unit and may outsource a portion of the project to a vendor in another country, for example, India. For competitive advantage, the outsourcing client may want to retain core activities such as analysis and testing while outsourcing commodity activities like programming. This arrangement provides for growth and agility while controlling costs for the outsourcing client. Several outsourcing vendors may be involved, although an analysis involving multiple vendors is beyond the scope of the paper.

The relationship between the outsourcing client and the outsourcing vendor is likely to be based on a contract.^{4, 5} Thus, the outsourcing client may not have a strong influence on the internal practices of the vendor. Persuading the vendor to follow certain development practices may be difficult. On the other hand, if the outsourcing vendor is a partner or collaborator in some respect, the client may have sufficient influence to initiate practices considered novel at the vendor site.

In adopting agile practices for outsourced projects, we need to consider factors such as the size and nature of the project, the cultural distance between the outsourcing client and the outsourcing vendor, along with the temporal, and the geographical distances.

Size and nature of the project. Today's software projects are developed in a dy-

Figure 1: Parties and Dynamics in Outsourced Software Development



namic, global, customer-focused, and competitive environment. Pure structure approaches are not suitable for dynamic environments.³ The nature of contemporary software development requires flexible and organic instead of overly hierarchical, mechanistic structures. The Agile Manifesto and its principles, therefore, are certainly relevant in this environment; however, to date agile development has been reported only for limited size projects and small teams.³ Beck² states: "Size clearly matters. You probably couldn't run an XP project with a hundred programmers. Not fifty. Nor twenty, probably. Ten is definitely doable." Agile development needs to be examined in the context of larger projects. Just as organizations cannot keep growing in an entrepreneurial mode and need formal coordination and control, agile development may need some degree of formality imposed for larger projects.

Cultural distance. Compared to discipline-based development, agile development denotes a significant shift in culture. There are no prescribed processes or tools; instead, values and principles govern agile development. Perhaps, its most important characteristic is the organizational and team culture as guided by a manifesto. Organizational culture is embedded in, and affected by, the larger national culture.⁸ Agile values, for instance, can fit in the less bureaucratic, more flexible American culture, but not necessarily in the more hierarchical and structured cultures found in Asia, Latin America, and certain parts of Europe. Work that

takes place over long distances will often involve communication engaging different national cultures.¹⁰ Business alliances and collaborations have been found to fail because of such differences.⁸ Organization literature has reported interesting "Catch-22" scenarios arising as a result of cultural clashes.

Temporal distance. It is obvious that temporal distance is also likely to mitigate communication, which is a critical ingredient of agile practice. When it is 9 am, or time to start work in New York, it is around 6:30 pm, or time to leave the office, in New Delhi. Even when collaborating parties are in similar time zones (such as those of the U.S. and Mexico), studies have shown that different orientations to time and schedules may lead to misunderstandings.

Geographical distance. This factor can greatly affect communication. Surely, the widespread use of email and other communication means have enhanced virtual communication. However, these mechanisms still largely facilitate "business" communication, not "organizational" communication. Business communication includes formalized and planned messages, codified in letters, memos, reports, Web sites, and advertising campaigns. Organizational communication includes business communication as well as informal, day-to-day interactions among organizational members.⁸ Agile practices, as currently prescribed, rely on organizational, not business communication. Although technologies such as videoconferencing and online chat address the geographical distance

Table 1. Support for Current Agile Values in Outsourcing Environment

Agile Manifesto Values	Advantages in an Outsourcing Environment	Problems in an Outsourcing Environment
Individuals and interactions vs. processes and tools.	There is some evidence that organizational communication may improve.	Various distances may impede communication and interactions. Both outsourcing and outsourced parties are likely to employ business (formal), instead of organizational (formal and informal), communication. Likely to conflict with the cultural values of the vendor party.
Working software vs. comprehensive documentation.	Software can be developed more quickly.	Documentation is essential for avoiding ambiguities arising because of lack of tacit knowledge.
Customer collaboration vs. contract negotiation.		The normal outsourcing practice is through contract negotiation.
Responding to change vs. following a plan.	More resources may be available to handle changes.	Vendor may not welcome changes unless there is additional compensation. Contract between client and vendor parties may not be easily renegotiated.

Interaction of Agile Principles with Outsourced Development

The Agile Manifesto lists 12 principles that should govern software development. The advantages, and the problems, found in subscribing to each principle in an outsourcing environment are listed in Table 2.

Outsourced development is generally governed by client-vendor contracts. Since organizational culture cannot be captured in a contract, a fundamental problem arises when employing some agile principles. For example, a contract cannot “welcome changing requirements” however, it can spell out the financial implications of the change requirements. Face-to-face communication is ruled out with outsourced development because of geographical distances; if high quality virtual meetings are a requirement, then someone needs to pay for installing additional equipment and providing the bandwidth. The practice of pair programming may be difficult to include in a contract. Pair programming is practically impossible between programmers at the client and the vendor end. The 40-hour week is a mirage in vendor companies; anecdotal evidence suggests that most developers in countries like India end up working 50-60 hours per week, which can lead to fatigue and burnout. Further, high turnover is a given, as developers continually scout for better pay and opportunities in a growing sector of the economy.

Modified Agile Practices for Outsourced Projects

The previous analysis indicates the use of agile practices in outsourced software projects is not a simple, straightforward issue. Nevertheless, agility is paramount in today's fast-paced, dynamic, and global environment, and the Agile Manifesto is a good starting point to analyze software practices. Successful organizations today are customer- and employee-focused, less hierarchical and more horizontal, more integrated with vendors,¹ responsive to change, and willing to adapt.

Yet, agile values and principles can be extreme and incongruous. In 1946, Herbert Simon¹² challenged the tenets of classical organizational theory proposed by Fayol, Gulick, and others by pointing out the inherent contradic-

problem, it is questionable if the resulting engagement satisfies the level expected in agile development.

Here, we explained the context for the various factors involved in evaluating agile practices in distributed settings. This contextual knowledge can be employed to scrutinize the agile values and principles in order to assess which ones hold up for larger projects in a distributed environment.

Interaction of Agile Values with Outsourced Development

The Agile Manifesto lists four fundamental values that should govern software development. Table 1 evaluates these values as applied in an outsourced project. Each value is qualified by the advantages it provides to outsourced development, and how the value may be mitigated in outsourced development.

Individuals and interactions vs. processes and tools. In co-located settings, agile practices improve organizational communication by facilitating informal interactions. In distributed development, the informal communication process is considerably broken down. For example, the assumption of tacit knowledge is challenged, and the pair programming practice is likely to be drastically affected. This value may not be welcome however, in regions where the national culture is characterized by high power distance, where the values tend toward hierarchy and structure.

Working software vs. comprehensive documentation. The basic premise of outsourcing is that it allows a project

to be done faster and cheaper. Thus, the goal of providing working software is likely to be achieved more quickly. However, documentation, too, is important in an outsourced project, given the realities and constraints of any agency agreement. As the project becomes bigger, one cannot rely on tacit knowledge alone.

Customer collaboration vs. contract negotiation. Transactional cost theory favors contractual agreements for outsourcing non-core activities of software development, especially given the low monitoring costs in today's global environment. Although some companies open their own offices at offshore sites and do not need a contract, the remote unit may not be large enough to provide the level of support needed given that the software activities that are typically outsourced (such as coding) need a large amount of personnel resources.

Responding to change vs. following a plan. The client-vendor contract cannot be renegotiated on an ad-hoc basis. If the customer proposes changes, especially at the later stages of a project, these changes will have to be passed on to the vendor. The vendor's response will depend on various factors such as the relationship with the outsourcing client, to what extent the agile culture is prevalent, the costs involved in responding to the changes and how these are recovered, and how much future business is expected. Whether one applies the agency theory, or just plain common sense here, it seems unlikely the vendor will absorb the costs arising from significant changes.

tions. The recently proposed Agile Manifesto has not been challenged so far, although some inconsistencies are evident. For example, if a software division decides to accept changes proposed late in the project, then it will have to ask the employees to work harder than the espoused 40-hour week. If the employees continue to work at the routine pace, then the project will get delayed, and this may not be acceptable to the customer. Similarly, if the division demands excellence from its employees, it will have to decide what to do with its below average employees, which goes against the agile principle that individuals are valued more than processes.

When the agile practices are subjected to the realities of outsourced development, the contradictions are especially evident, as suggested in Tables 1 and 2 in the previous section. Some of the values and principles are highly feasible, a few are infeasible, and the remainder need to be adjusted. Here, we consider each of these three categories and make recommendations in the outsourcing context. Since the principles capture the essence of the values, Table 2 is used to guide this discussion.

Feasible Agile Practices. The four principles termed as feasible in Table 2 are clearly viable.

Infeasible Agile Practices. The principles, “continuous attention to technical excellence and good design enhances agility” and “build projects around motivated individuals” are somewhat confusing at the outset. Software projects certainly need excellent design and technology skills, but they also need excellent requirements gathering, analysis, communication, team work, and other skills. Ideally, one would build projects around motivated individuals, but the “motivated” and “excellent” definitions are subjective and can be misinterpreted in organizations. For example, these definitions can be used to fire the bottom 10% of the personnel each year. A better practice would be to build motivated and capable individuals by providing training and development programs.

In an outsourced project, the client cannot control personnel excellence at the vendor end, unless the vendor has been acquired by the client. The client would obviously examine the capabilities and references of the vendor. If the

Table 2. Evaluation of Current Agile Principles in Outsourcing Environment

Agile Manifesto Principle	Advantages in an Outsourcing Environment	Problems in an Outsourcing Environment
Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.	Feasible because more resources are available.	
Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.	The lowering of costs because of outsourcing may provide some flexibility in managing changes.	Vendor may not accept late changes. Vendor may not agree to fixed price contract.
Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.	Feasible because more resources are available.	
Business people and developers must work together daily throughout the project.		Temporal and geographic distances may make it more difficult to work with the vendor.
Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.		Difficult to control motivation at the vendor's site. Approximately 49.99% of designers are below average.
The most efficient and effective method of conveying information to, and within, a development team is face-to-face conversation.		Practically impossible between the outsourcing client and vendor.
Working software is the primary measure of progress.	Feasible.	
Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.	Somewhat feasible.	Because of temporal distance, fatigue is inevitable if practices like pair programming are instituted.
Continuous attention to technical excellence and good design enhances agility.		Approximately 49.99% of designers are below average.
Simplicity – the art of maximizing the amount of work not done – is essential.	Feasible.	
The best architectures, requirements, and designs emerge from self-organizing teams.		Cultural disparities between the outsourcing client and vendor may inhibit self-organization.
At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.		Organizational culture may hamper this practice.

project is awarded to a vendor with an established record of technical excellence, then the client need not micro-manage the vendor. Eventually, the collaboration may lead to permeable structures and trusting relationships¹ and the vendor may be more open to instituting agile practices.

If the client is developing its own facility in an offshored area, there is some leeway in the selection of the top tier personnel. However, large scale hiring will be difficult with such an exacting constraint.

The principle, “the most efficient and effective method of conveying information to and within a development team is face-to-face conversation” when referring to communication between the client and the vendor in outsourced development is practically impossible.

Certainly, “disruptive technologies” may eventually overcome such barriers,⁶ but, as an example, high quality videoconferencing on workstations has not yet fully matured.

Modified Agile Practices. The principle, “welcome changing requirements, even late in development” is acceptable if the outsourcing client is ready to absorb the increased costs, and not try to pass it on to the vendor. Even then, the stipulation, “even late in development,” needs to be eliminated. Late stage changes are not feasible in large projects, which need a higher degree of accountability. The principle gives the customer *carte blanche* without the financial obligations.

The principle, “business people and developers must work together daily throughout the project,” is generally interpreted as the customer being located

at the developer's premises. This takes a broader meaning when three parties – customer, outsourcing client, and outsourcing vendor – are involved. If agile practices are applied literally, one may come up with a communication configuration as shown in Figure 2.

The recommended structure shown in Figure 3 is an altered configuration that maintains reasonable communication and feedback, but reduces the communication links required. The full lines indicate regular and defined communication links, while the dotted lines indicate informal and as-needed links. The customer representative is physically located at the outsourcing client in both configurations.

The vendor's team would likely resemble a programming team under a group leader, although there may also be design and other personnel. To facilitate communication while maintaining control, the outsourcing client needs to send an ambassador to the vendor's site. Typically, the ambassador would be someone who has spent a reasonable period of his or her life in the vendor country. The ambassador should be in daily communication with the programming leader, pro-

grammers, and designers at the vendor end. Similarly, there must be a primary contact on the outsourcing client side. This role is filled by the team coordinator or project leader, who forms the bridge between the customer and the outsourcing vendor.

The Agile Manifesto also puts stress on working software, so there is the need for an integration group. This group, consisting mainly of programmers, continually test, evaluate, and integrate software so that the programming group at the vendor's end gets daily feedback.

The customer, the ambassador, the coordinator, the programming group leader, and some members from the client's design team, and integration team should form an *agile team*. The agile team's purpose is to communicate on a daily basis through state-of-the-art videoconferencing in order to continually evaluate and monitor the project. This group can be the hub for daily planning, coordinating, communicating, monitoring, and control. However, this should not be the only network of communication, and informal communication should be encouraged among individuals, and between teams.

In light of the above discussion, the principle, “the best architectures, requirements, and designs emerge from self-organizing teams,” may need to be modified. Although the notion of a self-organizing team is sound, there is the need to incorporate accountability into agile development, especially if the size of the project is medium or large. This is done through the roles like the ones described in Figure 3. Large projects will need some degree of hierarchical structure to ensure accountability.

The principle, “at regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly,” requires an adaptive mindset, and given the cultural distances, may not be possible at the vendor end. The notion of self-organizing teams and adjusting behavior may be too much for organizational cultures built on high power distance (such as accepting power inequality) and high uncertainty avoidance. Organizational learning and change can take time.

Finally, the principle, “agile processes promote sustainable development; the sponsors, developers, and users should be able to maintain a constant pace indefinitely,” can be controlled at the client's end only. Unless the vendor company is owned by the client, the issue of constant pace should be left to the former. Given the agile practice of “welcoming changing requirements, even late in development,” this practice is easier said than done.

Conclusion

It is evident that agile practices will need to be modified for outsourced software projects that tend to be typically of medium or large size. To evaluate the claims made in the paper, researchers need to conduct empirical research based on case studies, surveys, and action research. Such research could lead to a hybrid approach that harmonizes agility and discipline, and adapts it based on the context and the environment. C

References

1. Ashkenas, R. N. *The boundaryless organization: Breaking the chains of organizational structure* (2nd Ed.). Jossey-Bass, 2002. San Francisco, CA.
2. Beck, K. *Extreme programming explained: Embrace change* (2nd Ed.). Addison-Wesley, Reading, MA, 2004.
3. Boehm, B. W., and Turner, R. *Balancing agility and discipline: A guide for the perplexed*. Addison-Wesley, Boston, 2003.

Figure 2.

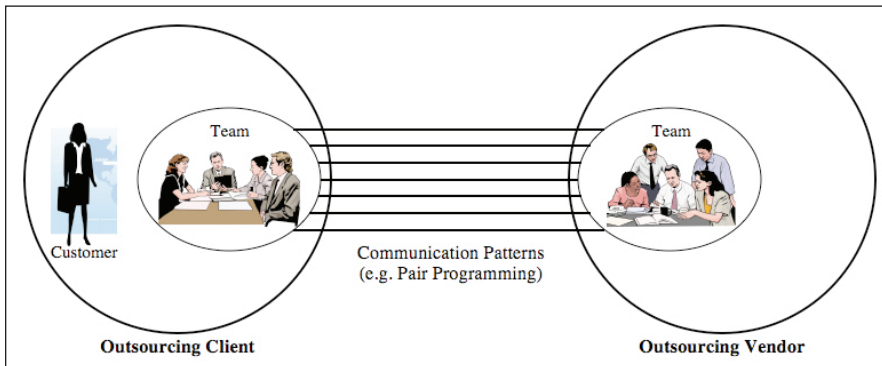
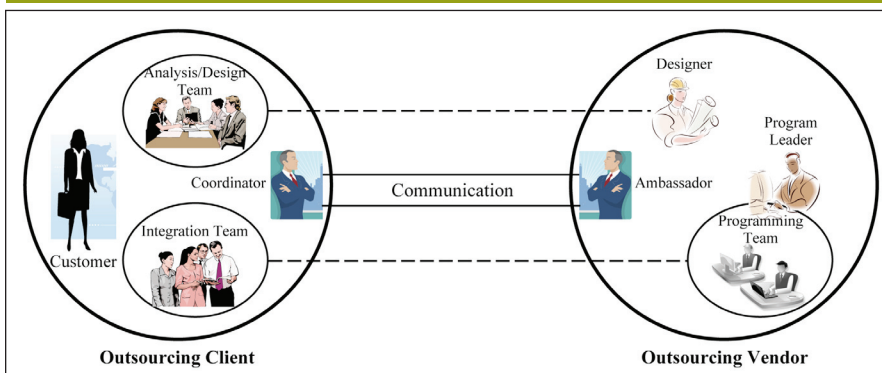


Figure 3.



4. Brown, D., and Wilson, S. *The black book of outsourcing: How to manage the changes, challenges, and opportunities*. John Wiley & Sons, Hoboken, N.J., 2005.
5. Cohen, L. and Young, A. *Multisourcing: Moving beyond outsourcing to achieve growth and agility*. Harvard Business School Press, Boston, 2006.
6. Hammer, M. and Champy, J. *Reengineering the Corporation: A manifesto for Business Revolution*. Harper Business, New York, 1993.
7. Holmstrom, H., Fitzgerald, B., Ågerfalk, P. J., and Conchúir, E. O. Agile practices reduce distance in global software development. *Information Systems Management*, 23, 3, (2006), 7-18.
8. Keyton, J. *Communication & organizational culture: A key to understanding work experiences*. Sage Publications, Thousand Oaks, CA, 2005.
9. Kircher, M., Jain, P., Corsaro, A., and Levine, D. Distributed extreme programming. *XP2001 Conference*, Sardinia, Italy, 2001.
10. Olson, J. S., and Olson, G. M. Culture surprises in remote software development teams. *ACM Queue* 1, 9, (2003), 52-59.
11. Sahay, S. Global software alliances: The challenge of standardization. *Scandinavian Journal of Information Systems*, 15, (2003), 3-21.
12. Simon, H.A. The proverbs of administration *Public Administration Review* 6, 1, Winter 1946, 53-67.

Dinesh Batra (Dinesh.Batra@business.fiu.edu) is a Knight-Ridder Research Professor in the College of Business Administration at the Florida International University. He is a co-author of the textbook *Object-Oriented Systems Analysis and Design*, Pearson Prentice-Hall.

© 2009 ACM 0001-0782/09/0900 \$10.00